

## Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

### INTRODUCTION

00 Overview & How the Web Works

### HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

### CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

### JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

### DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

# WS 101

## Web Systems and Technologies 1 — Complete Study Reference

Course: **WS 101**

Units: **3 units**

Year Level: **1st Year**

Type: **HTML/CSS/JS + PHP/MySQL + Laravel**

### CHAPTER 00

# Course Overview & How the Web Works

WS 101 starts by teaching you to build the **front-end of the web** — the structure (HTML) and visual presentation (CSS) of every website you have ever used — then continues into **server-side scripting**: PHP, MySQL, and the Laravel framework. Unlike CC 101's pure

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

theory, this is hands-on: every concept here is something you will type and immediately see rendered. This guide takes that further — wherever possible, you will see **live, actually-rendered output** next to the code that builds it, not just a screenshot.

### Critical Path — Layer 1 Foundation

WS 101 unlocks **WS 102** (Web Systems 2, Sem 2) directly, and indirectly feeds **HCI 101** (Human Computer Interaction, Year 2) and **IPT 102** (Integrative Programming 2, Year 3) which both require WS 101 + HCI 101 as prerequisites. Everything visual you build in your entire BSIT degree traces back to this course.

## The Client-Server Model

Every website you visit involves two computers talking to each other: a **client** (your phone/laptop, running a browser) and a **server** (a remote computer storing the website's files). This conversation happens in a clear request-response pattern.

**1**

**You type a URL or tap a link**

e.g. <https://qcu.edu.ph>

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### 2 Browser sends an HTTP Request

A formatted message asking the server: "send me the file at this address." DNS first resolves the domain name to an IP address (covered in CC 101 Ch. 7).

### 3 Server processes the request

The server locates the requested HTML file (or generates it dynamically) and prepares a response.

### 4 Server sends an HTTP Response

Contains a status code (200 OK, 404 Not Found, 500 Server Error) plus the actual HTML, CSS, JS, and asset files.

### 5 Browser renders the page

Parses HTML into a DOM tree, parses CSS into a CSSOM tree, combines them into a Render Tree, then paints pixels to your screen. This entire pipeline happens in milliseconds.

## How a Browser Renders a Page — The Pipeline

| Stage               | What Happens                                                                                                                  |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1. Parse HTML → DOM | Browser reads your HTML top to bottom and builds the <b>DOM</b> (Document Object Model) — a tree structure representing every |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Stage

### What Happens

element and its nesting relationship.

#### 2. Parse CSS → CSSOM

Browser reads all CSS rules and builds the **CSSOM** (CSS Object Model) — a tree of every style rule and what it applies to.

#### 3. Combine → Render Tree

DOM + CSSOM are merged into a Render Tree containing only visible elements with their final computed styles.

#### 4. Layout (Reflow)

Browser calculates the exact size and position of every element on the page based on the Render Tree.

#### 5. Paint

Browser fills in actual pixels — colors, text, images, borders — based on the Layout step.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Filipino Analogy — Client-Server Model

Think of ordering at a **carinderia counter**. You (client) place an order (HTTP Request) for "1 Adobo with rice" (a specific URL). The kitchen (server) prepares it and hands it back (HTTP Response) — either your food (200 OK) or "ubos na po" / sold out (404 Not Found). You then plate it yourself at your table exactly how you like it (the browser rendering the HTML/CSS into a visual page).

## HTML vs CSS vs JavaScript — The Three Pillars

| Layer      | Job                                     | Analogy                                | Covered In                    |
|------------|-----------------------------------------|----------------------------------------|-------------------------------|
| HTML       | Structure & content — the skeleton      | The walls, rooms, and frame of a house | WS 101 (this course)          |
| CSS        | Presentation & styling — the appearance | Paint, furniture, interior design      | WS 101 (this course)          |
| JavaScript | Behavior & interactivity — the logic    | Electricity, plumbing, automated doors | WS 101 (this course, Weeks 5– |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026-2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams &  
UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Layer

Job

Analogy

Covered  
In

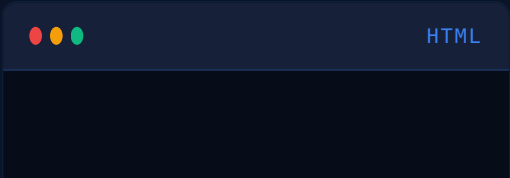
6) — then  
deepened  
in WS  
102

CHAPTER 01

HTML  
Document  
Structure &  
Semantic  
Elements

HTML (HyperText Markup Language) is not a programming language — it is a **markup language**. It does not compute logic; it describes the structure and meaning of content using tags. Every HTML document follows the same basic skeleton.

The Minimum HTML5  
Boilerplate



# Web Systems & Technologies 1

Complete Study Guide · AY 2026-2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, QCU!</h1>
</body>
</html>
```

### Line-by-Line Breakdown

| Line             | Purpose                                                                                                                                                              |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <!DOCTYPE html>  | Tells the browser this is an HTML5 document. Must be the very first line. Without it, browsers fall into "quirks mode" with inconsistent, legacy rendering behavior. |
| <html lang="en"> | The root element wrapping all content. The lang attribute helps screen readers and search engines                                                                    |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Line

### Purpose

identify the page language.

`<head>`

Contains metadata — information ABOUT the page that is NOT directly displayed (title, character encoding, linked CSS/ fonts, viewport settings).

`<meta charset="UTF-8">`

Sets character encoding so special characters (ñ, emoji, ₪) display correctly. UTF-8 is the universal standard — connects directly to Unicode from CC 101 Ch. 4.

`<meta name="viewport"...>`

Controls how the page scales on mobile devices. Without this,



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Line

### Purpose

your page would render zoomed-out and tiny on a phone screen like your GT 30 Pro.

`<title>`

Text shown in the browser tab and search engine results. Not visible on the page itself.

`<body>`

Contains ALL visible content — everything the user actually sees and interacts with.

## Semantic vs Non-Semantic Elements

**Semantic** elements clearly describe their meaning to both the browser and the developer.

**Non-semantic** elements ( `div` , `span` ) give no information about

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

their content — they are generic containers.

| Semantic Tag | Meaning                                             | Typical Use                              |
|--------------|-----------------------------------------------------|------------------------------------------|
| <header>     | Introductory content                                | Site logo, title, main navigation        |
| <nav>        | Navigation links                                    | Menu bar, links to other pages/ sections |
| <main>       | The dominant, unique content of the page            | Only ONE per page — the core content     |
| <section>    | A thematic grouping of related content              | "About Us" section, "Services" section   |
| <article>    | Self-contained, independently distributable content | A blog post, a news story, a forum post  |
| <aside>      | Content tangentially related to the main content    | Sidebar, related links, advertisement    |
| <footer>     | Closing content                                     | Copyright, contact info, social links    |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

## Live: Semantic Page Skeleton

This is what a semantic layout actually looks like rendered — each labeled box is a real semantic element, not a generic `div`.



LIVE PREVIEW

### Why Semantic HTML Matters

Screen readers (accessibility tools for visually impaired users) rely on semantic tags to navigate a page intelligently. Search engines (Google) also weigh semantic structure when ranking pages. A page built entirely from generic `<div>` tags works visually but is invisible in meaning to machines.

## The HTML Document Outline (Element Nesting)

### Concept

### Rule

Parent / Child

An element written inside another is its "child." The outer one is the "parent." `<body>` is the

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Concept

### Rule

parent of everything visible.

Nesting must be proper

Tags must close in the reverse order they opened:

`<p><strong>text</strong></p>` is correct.

`<p><strong>text</p></strong>` is broken/invalid.

Block-level elements

Start on a new line, take full available width by default.

Examples: `div`, `p`, `h1-h6`, `section`, `header`, `footer`, `ul`, `table`.

Inline elements

Do not start a new line, only take up as much width as their content needs.

Examples: `span`, `a`, `strong`, `em`, `img`, `label`.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &  
UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

## Forward Link — CC 102 / PF 101

The parent-child nesting concept you are learning right now is the exact same hierarchical thinking you will use for class inheritance in **PF 101** (Object-Oriented Programming, Year 2) — a child class inherits structure from its parent class, just like a nested HTML element inherits positioning context from its parent.

## CHAPTER 02

# Text, Lists, Links & Images

These are the elements that make up the actual readable content of nearly every web page — headings, paragraphs, text emphasis, lists, hyperlinks, and images.

## Headings & Paragraphs

HTML provides 6 levels of headings, `<h1>` through `<h6>`, in decreasing importance and size.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

There should only be **one** `<h1>` **per page** — it represents the main topic.

HTML

```
<h1>Main Title</h1>
<h2>Section Title</h2>
<h3>Subsection</h3>
<p>This is a normal
paragraph of text.</p>
<p><strong>Bold</strong> and
<em>italic</em> text.</p>
<blockquote>A quoted
passage.</blockquote>
```

LIVE  
PREVIEW

## Text Formatting Tags Reference

| Tag                         | Renders As | Semantic Meaning                                                           |
|-----------------------------|------------|----------------------------------------------------------------------------|
| <code>&lt;strong&gt;</code> | Bold       | Strong importance (screen readers emphasize vocally)                       |
| <code>&lt;b&gt;</code>      | Bold       | Visual bold only, no added importance — prefer <code>&lt;strong&gt;</code> |
| <code>&lt;em&gt;</code>     | Italic     | Emphasized stress — changes                                                |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Tag

Renders As

Semantic  
Meaning

meaning of a  
sentence

`<i>`

Italic

Visually  
offset text  
(foreign  
words,  
technical  
terms) — no  
emphasis

`<mark>`

Highlighted

Marked/  
highlighted  
relevance,  
like a  
highlighter  
pen

`<small>`

Smaller text

Fine print,  
disclaimers

`<del>`

Strikethrough

Deleted/  
removed text

`<code>`

Monospace

Inline code  
snippet

`<br>`

Line break

Forces a new  
line  
WITHOUT  
starting a  
new  
paragraph  
(self-closing,  
no closing  
tag)

`<hr>`

Horizontal  
line

Thematic  
break  
between  
content

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Tag      | Renders As            | Semantic Meaning                                                                                                                             |
|----------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
|          |                       | sections (self-closing)                                                                                                                      |
| <tt>     | Monospace             | Deprecated — HTML5 removed it. Use <code> instead. Still shown on lecture slides as "Monospaced Text."                                       |
| <big>    | Larger text           | Deprecated — removed in HTML5. Use CSS font-size instead.                                                                                    |
| <font>   | Sets face/ color/size | Deprecated — mixed styling into content. Use CSS instead: font-family, color, font-size properties replace its face, color, size attributes. |
| <center> |                       |                                                                                                                                              |



Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams &  
UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Tag

Renders As

Semantic  
Meaning

Centers

content

Deprecated  
— use CSS

`text-align:`  
`center`

(text) or

`margin: 0`

`auto` (block  
elements)  
instead.

Exam-Relevant but  
Obsolete — Physical vs  
Logical Style

Older HTML teaching  
material (including your  
lecture deck) splits text  
formatting into two  
categories: **Physical Style**

— tags that dictate exact  
visual appearance  
regardless of meaning

( `<b>` , `<i>` , `<tt>` , `<u>` ,  
`<s>` , `<big>` , `<small>` ,  
`<sub>` , `<sup>` ) — versus

**Logical Style** — tags that  
describe meaning and let  
the browser decide the  
visual ( `<em>` , `<strong>` ,

`<cite>` ). This terminology  
may appear in a quiz. In  
modern practice, the

"logical" category is exactly  
what the **Semantic**

**Meaning** column above is  
teaching you — `<strong>` /  
`<em>` over `<b>` / `<i>` for  
the same reason.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

# Lists

LIVE  
PREVIEW

```
<h3>Unordered</h3>
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>

<h3>Ordered</h3>
<ol>
  <li>Plan</li>
  <li>Build</li>
</ol>

<h3>Description</h3>
<dl>
  <dt>HTML</dt>
  <dd>Structure</dd>
</dl>
```

# Links — The Anchor Tag

| Attribute       | Purpose                                            | Example                   |
|-----------------|----------------------------------------------------|---------------------------|
| href            | Destination URL — required attribute               | href="https://qcu.edu.ph" |
| target="_blank" | Opens link in a new browser tab                    | target="_blank"           |
| rel="noopener"  | Security best practice paired with target="_blank" | rel="noopener noreferrer" |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Attribute

### Purpose

### Example

— prevents the new tab from accessing your original page

## Absolute vs Relative Paths

| Type     | Example                                                        | When to Use                                                                                                                      |
|----------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Absolute | <code>https://google.com</code>                                | Linking to an external website                                                                                                   |
| Relative | <code>about.html</code><br>or <code>../images/photo.jpg</code> | Linking to another file within your own project — works regardless of domain, essential for local testing in VS Code Live Server |

## Images

The `<img>` tag is **self-closing** (no separate closing tag) and requires two key attributes: `src` (the image file path) and `alt` (alternative text describing the

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

image for accessibility and for when the image fails to load).



HTML

LIVE  
PREVIEW

```
<figure>
  
  <figcaption>
    A scenic view
  </figcaption>
</figure>
```

### Gotcha Alert — Always Include alt Text

If you forget `alt`, screen readers announce the raw filename to visually impaired users — useless and unprofessional. If an image fails to load (broken link), the alt text displays in its place, telling the user what was supposed to be there.

## CHAPTER 03

# Tables & Forms

Tables display tabular (grid-based) data. Forms collect input

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

from users — the entry point for nearly every interactive feature on the web, from login pages to GCash transfers.

Tables

| Tag     | Purpose                                        |
|---------|------------------------------------------------|
| <table> | The container for the entire table             |
| <thead> | Groups the header row(s)                       |
| <tbody> | Groups the main data rows                      |
| <tr>    | Table Row                                      |
| <th>    | Table Header cell — bold, centered by default  |
| <td>    | Table Data cell — regular content cell         |
| colspan | Attribute — makes a cell span multiple columns |
| rowspan | Attribute — makes a cell span multiple rows    |

HTML

```
<table>
  <thead>
    <tr>
      <th>Subject</th>
      <th>Units</th>
```

LIVE  
PREVIEW

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
</tr>
</thead>
<tbody>
  <tr>
    <td>CC 101</td>
    <td>3</td>
  </tr>
  <tr>
    <td>WS 101</td>
    <td>3</td>
  </tr>
</tbody>
</table>
```

## Forms

The `<form>` tag wraps all input controls. Its `action` attribute specifies where submitted data goes; its `method` attribute specifies how ( `GET` appends data visibly to the URL, `POST` sends it invisibly in the request body — always use POST for passwords/ sensitive data).

### Common Input Types

| Type               | Renders As                     | Use Case                |
|--------------------|--------------------------------|-------------------------|
| <code>text</code>  | Single-line text box           | Name, username          |
| <code>email</code> | Text box with email validation | Email address — browser |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Type     | Renders As                                                         | Use Case                             |
|----------|--------------------------------------------------------------------|--------------------------------------|
|          |                                                                    | checks for @ format                  |
| password | Text box that masks input (.....)                                  | Passwords                            |
| number   | Numeric stepper input                                              | Age, quantity                        |
| checkbox | Square box, multiple selectable                                    | Agree to terms, multi-select options |
| radio    | Circle button, ONLY one selectable per group (same name attribute) | Gender, single-choice selection      |
| date     | Calendar picker                                                    | Birthdate                            |
| submit   | Clickable button that sends the form                               | "Submit" / "Login" button            |
| file     | File upload chooser                                                | Profile picture, document upload     |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

## Live: A Complete Registration Form

HTML

```
<form>
  <label for="nm">Name</label>
  <input type="text" id="nm"
    placeholder="Juan Dela Cruz">

  <label for="em">Email</label>
  <input type="email" id="em">

  <label>Section</label>
  <select>
    <option>SBIT1D</option>
    <option>SBIT1E</option>
  </select>

  <label>
    <input type="checkbox">
    I agree to terms
  </label>

  <button type="submit">
    Register
  </button>
</form>
```

### Always Pair label with for/id

The `<label for="nm">` attribute must match the input's `id="nm"` exactly. This connects them so clicking the label text also focuses/activates the input — critical for accessibility

LIVE  
PREVI  
—  
TAPPA



Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams &  
UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

and a noticeably better mobile  
tapping experience.

Validation Attributes

| Attribute                                       | Effect                                                                                                 |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>required</code>                           | Form cannot submit until this field has a value                                                        |
| <code>minlength</code> / <code>maxlength</code> | Sets minimum/maximum character count                                                                   |
| <code>min</code> / <code>max</code>             | Sets minimum/maximum numeric value (for <code>type="number"</code> )                                   |
| <code>pattern</code>                            | Custom regex pattern the input must match                                                              |
| <code>placeholder</code>                        | Faded hint text shown when the field is empty — NOT a substitute for a real <code>&lt;label&gt;</code> |

CHAPTER 04

Multimedia,  
Entities &

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

# Global Attributes

This chapter rounds out HTML with embedding audio/video, special characters, and the universal attributes available on every HTML element.

## Audio & Video



HTML

```
<video controls width="320">
  <source src="clip.mp4" type="video/mp4" />
  Your browser does not support the video element.
</video>
```

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg" />
  Your browser does not support the audio element.
</audio>
```

### Attribute

### Effect

**controls**

Shows play/pause/volume/seek UI controls

**autoplay**

Starts playing automatically on page load (browsers often block this for sound; use with **muted**)

**loop**

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Attribute

### Effect

Restarts playback automatically when it ends

**muted**

Starts with sound off

**poster**

(video only) Image shown before playback starts

## The iframe Element

An `<iframe>` embeds an entirely separate HTML document inside the current page — like a window into another website. This is literally the technique used to build every "live preview" box throughout this guide.



HTML

```
<iframe
  src="https://www.youtube.com/
  width="320" height="180">
</iframe>
```

## Frames & Frameset — Legacy Predecessor to iframe

Before `<iframe>` existed, HTML used `<frameset>` to split the ENTIRE browser window into multiple independent panels, each loading a separate HTML

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

file. Your lecture deck still teaches this.

HTML

```
<html>
<head><title>HTML Frames</title>
<frameset rows="10%,80%,10%">
  <frame name="top" src="AboutUs">
  <frame name="main" src="Galler">
  <frame name="bottom" src="Cont">
</frameset>
</html>
```

| Tag/<br>Attribute | Purpose                                                                                       |
|-------------------|-----------------------------------------------------------------------------------------------|
| <frameset>        | Replaces <body> entirely — splits the window using rows or cols (e.g. rows="10%, 80%,10%" )   |
| <frame />         | One panel inside a frameset — src points to the HTML file it loads, name lets links target it |
| <noframes>        | Fallback content for browsers that don't support frames                                       |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Deprecated — Do Not Use in Real Projects

`<frameset>` is obsolete in HTML5: it breaks bookmarking, browser back/forward, SEO, and accessibility, since the URL never reflects which content is showing. `<iframe>` (above) replaced it — it embeds one document inside a normal page instead of replacing the whole page. Know `<frameset>` / `<frame>` / `<noframes>` for the exam; use `<iframe>`, or better, CSS layout, for actual work.

## HTML Entities — Special Characters

Some characters have special meaning in HTML (like `<` and `>` for tags) and cannot be typed directly into content. **Entities** are escape codes that represent them safely.



HTML

```
&lt; &gt; &amp;
&copy; &hearts;
&nbsp; &quot;
```

LIVE  
PREVIEW

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams &  
UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Entity Code | Renders | Meaning |
|-------------|---------|---------|
|-------------|---------|---------|

|         |         |                                                       |
|---------|---------|-------------------------------------------------------|
| &lt;    | <       | Less than                                             |
| &gt;    | >       | Greater than                                          |
| &amp;   | &       | Ampersand                                             |
| &quot;  | "       | Quotation mark                                        |
| &nbsp;  | (space) | Non-breaking space — prevents line wrap at that point |
| &copy;  | ©       | Copyright symbol                                      |
| &#8369; | ₱       | Philippine Peso sign (numeric entity)                 |

Global Attributes

These attributes can be applied to virtually ANY HTML element, regardless of tag type.

| Attribute | Purpose                                                                                         |
|-----------|-------------------------------------------------------------------------------------------------|
| id        | A UNIQUE identifier — only one element per page may have a given id. Used for CSS targeting, JS |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Attribute

### Purpose

selection, and anchor links ( `#overview` ).

#### class

A reusable label — MANY elements can share the same class. The primary way to target groups of elements with CSS.

#### style

Inline CSS applied directly to that one element. Convenient but considered poor practice for real projects (mixes structure with presentation).

#### title

Tooltip text shown on mouse hover.

#### data-\*

Custom data attributes for storing extra information accessible via JavaScript, e.g. `data-student-id="12-3456"`.

#### hidden

Hides the element completely from the rendered page.

## id vs class — The Most Important Distinction

id

class

Uniqueness

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

|              | id                                                                 | class                                                             |
|--------------|--------------------------------------------------------------------|-------------------------------------------------------------------|
|              | ONE per page                                                       | Reusable on MANY elements                                         |
| CSS Selector | #myId<br>{ }                                                       | .myClass<br>{ }                                                   |
| Specificity  | Higher (overrides class rules)                                     | Lower                                                             |
| Best for     | One specific unique element (a particular header, a single button) | A reusable style applied to many elements (all "card" components) |

## HTML Comments

HTML

```
<!-- This is a comment. It is in  
to the user but visible in  
<p>Visible text</p>
```

Comments are useful for leaving notes for yourself or teammates, and for temporarily disabling a block of code without deleting it.



Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

CSS  
Fundamentals

CSS (Cascading Style Sheets) controls how HTML elements LOOK — colors, spacing, fonts, layout. The "Cascading" in the name refers to the rules that determine which style wins when multiple rules target the same element.

History of CSS (exam trivia)

CSS was first proposed by **Håkon Wium Lie** on October 10, 1994. The first W3C CSS Recommendation (**CSS1**) was released in 1996, with **Bert Bos** as co-author — he's regarded as CSS's co-creator alongside Lie.

Three Ways to Add CSS

| Method | Where It Lives                                           | Example                                  | Besides Practical       |
|--------|----------------------------------------------------------|------------------------------------------|-------------------------|
| Inline | Directly on the element via <code>style</code> attribute | <code>&lt;p style="color:red"&gt;</code> | — h<br>spe<br>han<br>ma |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams &  
UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Method   | Where It Lives                                          | Example                                                     | Besides Practical                                  |
|----------|---------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------|
| Internal | Inside a <code>&lt;style&gt;</code> tag in the document | <code>&lt;style&gt;p{color:red}&lt;/style&gt;</code>        | mix<br>con                                         |
|          |                                                         |                                                             | sm<br>sing<br>pag<br>den                           |
| External | A separate file linked via <code>&lt;link&gt;</code>    | <code>&lt;link rel="stylesheet" href="style.css"&gt;</code> | pra<br>—<br>reu<br>ac<br>mul<br>pag<br>clea<br>sep |

CSS Syntax Anatomy

```
selector {  
  property: value;  
  property: value;  
}  
  
/* Real example: */  
h1 {  
  color: #3B82F6;  
  font-size: 28px;  
}
```

| Part     | Role                                                        |
|----------|-------------------------------------------------------------|
| Selector | WHICH element(s) to style ( <code>h1</code> in the example) |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Part

### Role

Declaration Block

Everything inside the curly braces  
`{ }`

Property

WHAT aspect to change ( `color` , `font-size` )

Value

The setting applied to that property  
( `#3B82F6` , `28px` )

Declaration

One full property:value; pair

## CSS Selectors — Full Reference

### Selector

### Syntax

### Targets

Universal

`*`

EVERY element on the page

Element / Type

`p`

Every `<p>` tag

Class

`.card`

Every element with `class="card"`

ID

`#header`

The one element with `id="header"`

Descendant

`nav a`

Any `<a>` nested ANYWHERE

Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Selector       | Syntax                          | Targets                                                                               |
|----------------|---------------------------------|---------------------------------------------------------------------------------------|
|                |                                 | inside a <code>&lt;nav&gt;</code>                                                     |
| Child          | <code>nav &gt; a</code>         | Only <code>&lt;a&gt;</code> tags that are DIRECT children of <code>&lt;nav&gt;</code> |
| Grouping       | <code>h1, h2, h3</code>         | Applies the same rule to a listed selector                                            |
| Attribute      | <code>input[type="text"]</code> | Elements with a matching attribute value                                              |
| Pseudo-class   | <code>a:hover</code>            | An element in a specific STATE (hovered, focused, first child)                        |
| Pseudo-element | <code>p::first-line</code>      | A specific PART of an element content                                                 |

Common Pseudo-Classes

| Pseudo-class        | Triggers When                                                                    |
|---------------------|----------------------------------------------------------------------------------|
| <code>:hover</code> | Mouse pointer is over the element (desktop only — no true hover on touchscreens) |
| <code>:focus</code> | Element is currently selected/active (e.g.                                       |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Pseudo-class

### Triggers When

a clicked/tapped input field)

`:active`

Element is being clicked/tapped at this exact moment

`:first-child`

Element is the first child of its parent

`:last-child`

Element is the last child of its parent

`:nth-child(n)`

Element is the nth child — supports formulas like `:nth-child(2n)` for every even item

## Live: Selector Specificity in Action

Same `<p>` tag, three competing rules. Watch which one wins.



CSS

```
p { color: gray; }           /* weakest */

.note { color: orange; } /* medium */

#alert { color: red; }      /* strongest */

<p id="alert"
  class="note">
  Which color wins?
</p>
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

## The Cascade — Specificity Hierarchy

When multiple rules target the same element, CSS resolves the conflict using this priority order, from weakest to strongest:

1

### Element selectors

Lowest specificity. e.g. `p { }`

2

### Class, attribute, and pseudo-class selectors

e.g. `.note { }`, `[type="text"] { }`, `:hover { }`

3

### ID selectors

e.g. `#alert { }` — beats any number of class selectors

4

### Inline styles

e.g. `style="color:red"` directly on the tag — beats all selectors in external/internal CSS

5

### !important

Highest possible priority — overrides EVERYTHING, including inline styles. Use only as an absolute last resort; heavy use makes CSS unmaintainable.

### Gotcha Alert — When Specificity Ties

If two rules have IDENTICAL specificity (e.g. two class selectors), the one written LATER in the CSS file wins. This is the actual "cascading" behavior —

**INTRODUCTION**

00 Overview &amp; How the Web Works

**HTML**

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

**CSS**

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

**JAVASCRIPT**

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events &amp; Embedding JS

**DATABASES & BACKEND**13 Relational Model, ER Diagrams &  
UML

14 SQL &amp; Connecting PHP to MySQL

**LARAVEL FRAMEWORK**

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

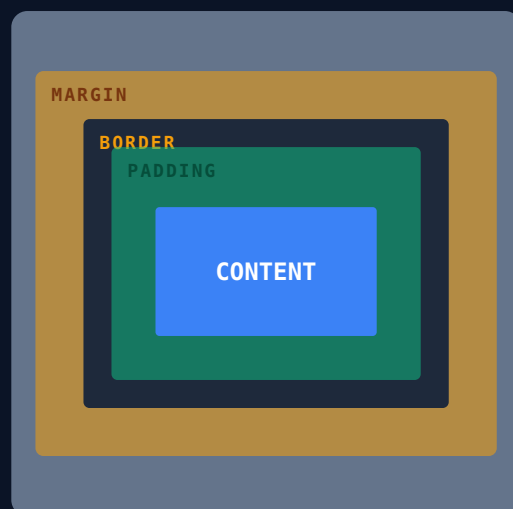
order matters as a tiebreaker.

## CHAPTER 06

# The Box Model, Display & Positioning

Every single HTML element is, fundamentally, a rectangular box. The Box Model describes the four layers that make up that box. This is the single most exam-tested CSS concept in WS 101 — master this diagram completely.

## The Box Model — Four Layers



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Layer

### What It Is

### CSS Property

#### Content

The actual text, image, or content itself

`width`,  
`height`

#### Padding

Transparent space INSIDE the border, cushioning the content

`padding`

#### Border

A visible (or invisible) line wrapping the padding

`border`

#### Margin

Transparent space OUTSIDE the border, pushing away other elements

`margin`



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Filipino Analogy — The Box Model

Think of a **bilao of food at a kamayan feast**. The Content is the actual rice and viands. The Padding is the banana leaf lining directly around the food. The Border is the rim of the bilao tray itself. The Margin is the empty table space between your bilao and the next person's bilao — keeping servings from touching.

## box-sizing — The Property That Changes Everything

| Value                              | Behavior                                                                                                                                             |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>content-box</code> (default) | Width/height apply ONLY to content. Padding and border are ADDED on top, making the box visually bigger than the set width.                          |
| <code>border-box</code>            | Width/height include padding and border. The box stays exactly the size you set — padding/border eat into the content space instead of expanding it. |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

CSS

```
* {  
  box-sizing: border-box; /* near
```

### Industry Standard Practice

Almost every professional CSS file starts with `*` `{ box-sizing: border-box; }`. It makes sizing predictable — a 300px box stays exactly 300px no matter how much padding or border you add, which matches how most developers intuitively expect sizing to work.

## Display Property

| Value               | Behavior                                                                            | Examples                                                                   |
|---------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>block</code>  | Starts on a new line, fills full available width                                    | <code>div</code> , <code>p</code> , <code>h1</code> , <code>section</code> |
| <code>inline</code> | Flows within text, only as wide as content, CANNOT set width/height/vertical margin | <code>span</code> , <code>a</code> , <code>strong</code> , <code>em</code> |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Value

### Behavior

### Examples

`inline-block`

Flows within text like inline, BUT can have width/ height/ margin set

Common for nav links, buttons in a row

`flex`

Turns the element into a flex container — children arrange via Flexbox rules (Chapter 08)

Modern layout standard

`grid`

Turns the element into a grid container (Chapter 09)

2D layout systems

`none`

Completely removes the element from the page — takes up zero space

Hiding modals, toggling menus

## Live: Block vs Inline vs Inline-Block



LIVE PREVIEW

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

## Positioning

### Value

### Behavior

**static**  
(default)

Normal document flow. **top/left/right/bottom** have NO effect.

**relative**

Stays in normal flow, but can be nudged via **top/left/right/bottom** RELATIVE to where it would normally sit. Leaves a gap behind where it would have been.

**absolute**

Removed from normal flow entirely. Positioned relative to its nearest ancestor that has **position: relative** (or the page if none exists).

**fixed**

Removed from flow, positioned relative to the BROWSER WINDOW. Stays in place even when scrolling — used for sticky headers/floating buttons.

**sticky**

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Value

### Behavior

Hybrid: acts like `relative` until the page scrolls past a threshold, then "sticks" like `fixed`.

## Live: Relative + Absolute Positioning

 LIVE PREVIEW

### Key Rule — absolute Needs a relative Parent

An absolutely positioned element anchors itself to the nearest ANCESTOR with `position: relative` set. This is exactly how the red "NEW" badge above stays pinned to the top-right corner of its gray container, instead of the entire page.

## CHAPTER 07

# Typography, Color & Backgrounds

This chapter covers how to control text appearance, the four

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

ways to specify color in CSS, and how to style element backgrounds.

Typography Properties

| Property    | Controls                               | Example Values                                                                   |
|-------------|----------------------------------------|----------------------------------------------------------------------------------|
| font-family | Typeface                               | Arial, sans-serif — always list a fallback in case the first font is unavailable |
| font-size   | Text size                              | 16px, 1.2rem                                                                     |
| font-weight | Boldness                               | normal (400), bold (700), or numeric 100–900                                     |
| font-style  | Italics                                | normal, italic                                                                   |
| line-height | Vertical spacing between lines of text | 1.5 (unitless multiplier is best practice)                                       |
| text-align  | Horizontal alignment                   | left, center, right, justify                                                     |

16 Controllers, Blade S, Migrations

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Method     | Syntax                                                           | Notes                                                                                                   |
|------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
|            |                                                                  | white, <code>#000000</code> black.                                                                      |
| RGB / RGBA | <code>rgb(59,130,246)</code> / <code>rgba(59,130,246,0.5)</code> | Red, Green & Blue values 0–255. The "A" (Alpha) controls transparency. 0 = invisible, 1 = fully opaque. |
| HSL        | <code>hsl(217,91%,60%)</code>                                    | Hue (0–360° on a color wheel), Saturation & Lightness % Most intuitive for humans to hand-tune          |

Live: Same Blue, Four Ways

**Named**  
royalblue

**Hex**  
#3B82F6

**RGB**  
rgb(59,130,246)

**HSL**  
hsl(217,91%,60%)



Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Backgrounds

Property

Controls

background-color

Solid fill color

background-image

Image or gradient:  
url('photo.jpg')  
or linear-gradient(...)

background-size

cover (fills, may crop), contain (fits fully, may letterbox)

background-position

Where the image is anchored, e.g. center, top right

background-repeat

repeat (default, tiles), no-repeat (shows once)

Live: Gradient Backgrounds

LIVE PREVIEW

Borders & Border Radius



CSS

```
.card {  
  border: 2px solid #3B82F6; />  
  border-radius: 12px;  
}  
.circle {
```

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
border-radius: 50%;  
}
```

## CHAPTER 08

# CSS Flexbox

Flexbox (Flexible Box Layout) is the modern standard for arranging elements in a single row or column, with powerful alignment and spacing control. It solved decades of layout hacks (floats, table-based layouts) and is the **priority layout system** for WS 101 — master this chapter thoroughly.

## Activating Flexbox

```
.container {  
  display: flex; /* this one li  
}
```

Once `display: flex` is set on a parent, all of its direct children automatically become "flex items" arranged in a row by default — no floats, no manual positioning needed.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

## flex-direction

Sets the **main axis** — the primary direction items flow along.

 LIVE PREVIEW

## justify-content — Main Axis Alignment

Controls spacing of items along the main axis (horizontal, for default row direction).

 LIVE PREVIEW

## align-items — Cross Axis Alignment

Controls alignment PERPENDICULAR to the main axis (vertical, for default row direction).

 LIVE PREVIEW

## flex-wrap

By default, flex items try to squeeze onto ONE line, shrinking if necessary. `flex-wrap: wrap` allows items to flow onto multiple lines instead.

 LIVE PREVIEW

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Flexbox Property  
Quick Reference

| Property                     | Applied To | Purpose                                                      |
|------------------------------|------------|--------------------------------------------------------------|
| <code>display: flex</code>   | Parent     | Activates flexbox for direct children                        |
| <code>flex-direction</code>  | Parent     | row / column / row-reverse / column-reverse                  |
| <code>justify-content</code> | Parent     | Aligns items along the main axis                             |
| <code>align-items</code>     | Parent     | Aligns items along the cross axis                            |
| <code>flex-wrap</code>       | Parent     | nowrap (default) / wrap                                      |
| <code>gap</code>             | Parent     | Sets uniform spacing BETWEEN items, no manual margins needed |
| <code>flex-grow</code>       | Child      |                                                              |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Property

Applied  
To

Purpose

How  
much an  
item  
expands  
to fill  
leftover  
space  
(ratio-  
based)

flex-  
shrink

Child

How  
much an  
item  
shrinks  
when  
space is  
tight

align-  
self

Child

Overrides  
align-  
items  
for ONE  
specific  
item

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Filipino Analogy — Flexbox

Think of **jeepney seating**.

**flex-direction** is whether passengers sit facing each other in a row (row) or single-file (column).

**justify-content** is how passengers space themselves along the bench — bunched at the front (flex-start), evenly spread with gaps (space-between).

**align-items** is whether everyone aligns to the same height on the bench (stretch) or sits at varied heights (flex-start/center/flex-end).

## CHAPTER 09

# CSS Grid, Units & Responsive Design

This final chapter covers two-dimensional layout with Grid, the measurement units CSS uses, and how to make pages adapt across phone, tablet, and desktop screens — essential

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

since you build and test everything on your GT 30 Pro.

## CSS Grid — Two-Dimensional Layout

Where Flexbox arranges items along ONE axis (row OR column), Grid arranges items along BOTH axes simultaneously — true rows AND columns at once.



CSS

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 10px;  
}
```

### Live: 3-Column Grid Gallery



LIVE PREVIEW

### Live: Grid with Spanning Items

An item can span multiple columns or rows using `grid-column` / `grid-row` — useful for hero banners or featured cards inside a grid.



CSS

```
.featured {  
  grid-column: span 2;  
}
```

LIVE  
PREVIEW

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
/* spans 2 columns wide */  
}
```

Flexbox vs Grid —  
When to Use Which

|                 | Flexbox                                                     | Grid                                                         |
|-----------------|-------------------------------------------------------------|--------------------------------------------------------------|
| Dimensions      | One-dimensional (row OR column)                             | Two-dimensional (rows AND columns together)                  |
| Best for        | Navbars, button groups, centering content, small components | Full page layouts, photo galleries, dashboards               |
| Content-driven? | Yes — sizing often driven by content                        | No — layout-driven, content fits into a predefined structure |

CSS Units

| Unit | Type     | Relative To                      | Common Use                     |
|------|----------|----------------------------------|--------------------------------|
| px   | Absolute | Fixed pixel count, never changes | Borders, small precise details |



## 16    Controllers, Blade &amp; Migrations

| Unit | Type     | Relative To                                                  | Common Use                                              |
|------|----------|--------------------------------------------------------------|---------------------------------------------------------|
| %    | Relative | Parent element's size                                        | Fluid widths                                            |
| em   | Relative | The font-size of the CURRENT element (compounds when nested) | Padding/spacing that should scale with text             |
| rem  | Relative | The ROOT ( <html> ) font-size only — does not compound       | Font sizes, modern best practice for consistent scaling |
| vw   | Relative | 1% of Viewport Width                                         | Full-bleed hero sections, responsive type               |
| vh   | Relative | 1% of Viewport Height                                        | Full-screen sections ( 100vh = exactly one screen tall) |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Gotcha Alert — em vs rem

**em** compounds: if a parent has `font-size: 1.2em` and a child also has `1.2em`, the child's ACTUAL size is  $1.2 \times 1.2 \times$  base size — sizes spiral unpredictably when nested. **rem** always refers to the root font-size only, making it far more predictable. Most professionals default to **rem** for this reason.

## Responsive Design & Media Queries

A **media query** applies CSS rules ONLY when certain conditions are met — most commonly, screen width.



CSS

```
/* Mobile-first base styles (applies to all screens) */
.container {
  display: flex;
  flex-direction: column;
}

/* Tablet and up */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}

/* Desktop and up */
@media (min-width: 1024px) {
```

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

```
.container {  
    max-width: 1200px;  
    margin: 0 auto;  
}  
}
```

Common Breakpoints

| Device Class | Typical Breakpoint                                       |
|--------------|----------------------------------------------------------|
| Mobile       | 0 – 767px (your GT 30 Pro's browser viewport falls here) |
| Tablet       | 768px – 1023px                                           |
| Desktop      | 1024px and above                                         |

Mobile-First vs Desktop-First

| Approach                   | Strategy                                                                                                             | Why It Matters                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Mobile-First (recommended) | Write base CSS for small screens, then use <code>min-width</code> media queries to ADD complexity for larger screens | Forces simpler, performance-friendly design first. Matches how most users now browse — on phones. |
| Desktop-First              | Write base CSS for large                                                                                             | Older approach, often results                                                                     |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Approach

Strategy

Why It Matters

screens, then use `max-width` media queries to REMOVE/adjust for smaller screens

in bloated mobile experiences carrying unnecessary desktop styling

Forward Link — HCI 101 & WS 102

Everything in this chapter is the direct foundation for **HCI 101** (Human Computer Interaction, Year 2 Sem 2, requires WS 102) where responsive, user-centered design becomes the entire focus. The JavaScript chapters that follow build interactivity directly on top of these responsive structures — and **WS 102** (Sem 2) deepens that further with more advanced client-side techniques.

Quick Reference —  
Building Your First Real Page

1

Structure first

Write complete, semantic HTML with no styling at all. Confirm content order makes sense unstyled.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

## 2 Reset/normalize basics

Start your CSS with `* { box-sizing: border-box; margin: 0; padding: 0; }`

## 3 Layout with Flexbox/Grid

Establish the overall page skeleton positioning before touching colors or fonts.

## 4 Style details

Typography, colors, backgrounds, borders — the visual polish layer.

## 5 Test responsiveness

Use VS Code Live Server and resize the browser (or open dev tools device toolbar) to check mobile/tablet/desktop breakpoints.

## CHAPTER 10

# JavaScript Fundamentals — Variables, Data & Strings

JavaScript is the third pillar of the web — where HTML gives structure and CSS gives appearance, JavaScript gives **behavior**. It is a **scripting language**: it runs inside the browser, reacting to what the

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

user does and modifying the page live, without a page reload.

What JavaScript Can Do

| Capability                         | Example                                                               |
|------------------------------------|-----------------------------------------------------------------------|
| Scrolling messages                 | Marquee-style ticker text                                             |
| Animation & dynamic images         | Image sliders, hover-triggered swaps                                  |
| Data input forms                   | Client-side validation before submitting to a server                  |
| Pop-up windows                     | <code>alert()</code> , <code>confirm()</code> , <code>prompt()</code> |
| Interactive surveys/questionnaires | Live score calculation, conditional follow-up questions               |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Why It's Called a "Scripting Language"

JavaScript is interpreted line-by-line by the browser at runtime, is always visible in the browser's "View Source"/DevTools, runs by default in every modern browser, and behaves identically across platforms. Your lecture deck lists this combination as JS's "unique features": script-based output, code visibility, full functionality, simplicity, default scripting support, and cross-platform support.

## Variables

A variable is a named container for a value. The lecture deck declares variables with `var` — the original JavaScript keyword. Modern code favors `let` / `const`, which behave similarly but scope more predictably; both are worth recognizing.

## Naming Conventions

| Valid                    | Invalid                  | Why                                  |
|--------------------------|--------------------------|--------------------------------------|
| <code>idnumber</code>    | <code>id number</code>   | No spaces allowed in a variable name |
| <code>Customer_ID</code> | <code>Customer.ID</code> |                                      |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026-2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Valid

### Invalid

### Why

No periods — use underscores instead

SY2016

2016

Cannot start with a digit

\_Continue

Continue

Reserved keywords need a prefix — `continue` is a JS keyword

## Global vs Local Variables

### Type

### Scope

Global Variable

Declared outside any function — accessible anywhere in the script

Local Variable

Declared inside a function — only accessible within that function

## Data Types

### Type

### Example

Alphabetical

"A"

Integer / Floating-point

10 , 3.14



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Type

### Example

Boolean /  
Logical

`true` , `false`

String

`"Welcome to QCU"`

Null value

`null` —  
deliberately  
empty

Form input often arrives as text even when it looks numeric. Two built-in functions convert it:

### Function

### Converts To

### Example

`parseInt()`

Whole  
number

`parseInt("25")` →  
`25`

`parseFloat()`

Decimal  
number

`parseFloat("3.14")`  
→ `3.14`

## Live: Variables, Data Types & Conversion



HTML

```
<script>
var schoolName = "QCU";
var unitsText = "3";
var units = parseInt(unitsText);
var isEnrolled = true;

document.write(
  schoolName + " - " + units +
  " units - Enrolled: " + isEnrolled
)
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026-2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
);  
</script>
```

## Literals

A **literal** is a fixed value written directly into code, as opposed to a variable that can change.

| Type            | Example                              |
|-----------------|--------------------------------------|
| Numeric Literal | <code>var width = 3;</code>          |
| String Literal  | <code>var schoolName = "QCU";</code> |

## Strings

A string can be created as a simple value, or explicitly as a String object with `new` :

```
JS  
  
strTest = "Welcome to QCU";  
// or, as an object:  
strTest = new String("Welcome to QCU");
```

## Live: String Concatenation

```
HTML  
  
<script>  
var strTest1 = "Welcome to QCU.";   
var strTest2 = "Enjoy your stay.";   
var strBoth = strTest1 + " " + strTest2;
```

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
document.write(strBoth);  
</script>
```

Useful String Methods

| Method/Property             | Effect                          | Example                                                |
|-----------------------------|---------------------------------|--------------------------------------------------------|
| <code>.length</code>        | Property<br>— counts characters | <code>"Welcome to QCU".length</code> → <code>14</code> |
| <code>.toUpperCase()</code> | Converts to ALL CAPS            | <code>strTest.toUpperCase</code>                       |
| <code>.toLowerCase()</code> | Converts to all lowercase       | <code>strTest.toLowerCase</code>                       |

Expressions & Operators

| Operator       | Meaning                                       | Example                                        |
|----------------|-----------------------------------------------|------------------------------------------------|
| <code>+</code> | Addition (numbers) or concatenation (strings) | <code>result = 5 + 7;</code> → <code>12</code> |
| <code>-</code> | Subtraction                                   | <code>score = score - 1;</code>                |
| <code>*</code> | Multiplication                                | <code>total = quantity * price;</code>         |
| <code>/</code> | Division                                      | <code>average = sum / 4;</code>                |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Operator        | Meaning                           | Example                           |
|-----------------|-----------------------------------|-----------------------------------|
| <code>%</code>  | Modulo — remainder after division | <code>remainder = sum % 4;</code> |
| <code>++</code> | Increment by 1                    | <code>tries++;</code>             |
| <code>--</code> | Decrement by 1                    | <code>total--;</code>             |

## Arrays

An array is a special variable that holds MULTIPLE values under one name, accessed by a numeric index starting at `0`.

JS

```
var score = new Array();
score[0] = 39;
score[1] = 40;
score[2] = 100;
score[3] = 49;

// modern shorthand – same result:
var score2 = [39, 40, 100, 49];
```

LIVE  
PREV

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Forward Link — CC 102

Arrays, variables, and data types here are the same concepts you're building in **CC 102** (Java) — just different syntax. A JavaScript array and a Java array both use zero-based indexing and store an ordered collection under one name.

## CHAPTER 11

# Control Structures, Functions & Objects

With data covered, this chapter covers how JavaScript makes DECISIONS (conditionals), REPEATS actions (loops), REUSES code (functions), and ORGANIZES related data (objects).

## Flow of Control & Flowcharts

A **flowchart** diagrams the order operations execute in. Ovals

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

mark start/end, parallelograms mark input/output, and diamonds mark decision points with Yes/No branches.

### Deck Example

Lecture deck's flowchart:  
Leave home → Check time  
→ (diamond) Before 7am? →  
Yes: Take bus / No: Take subway → Reach school.  
This is exactly the logic an `if/else` statement encodes below.

## Conditional

### Statements — The If Statement



JS

```
var x = 10;  
if (x == 10) {  
    window.alert("The number is 10")  
}
```

### Conditional Expressions

| Operator         | Meaning                              | Example               |
|------------------|--------------------------------------|-----------------------|
| <code>==</code>  | Equal to (value only, no type check) | <code>x == 10</code>  |
| <code>===</code> | Strictly equal (value and type)      | <code>x === 10</code> |

Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Operator | Meaning                                              | Example             |
|----------|------------------------------------------------------|---------------------|
|          | AND<br>type) —<br>modern<br>best<br>practice         |                     |
| < / >    | Less<br>than /<br>greater<br>than                    | y < z               |
| &&       | Logical<br>AND —<br>both<br>sides<br>must be<br>true | x > 0<br>&& x < 10  |
|          | Logical<br>OR — at<br>least<br>one side<br>true      | x == 1<br>   x == 2 |

Live: If/Else Grade Checker

HTML

```
<script>
var grade = 78;
if (grade >= 75) {
  document.write("PASSED");
} else {
  document.write("FAILED");
}
</script>
```

LIVE  
PREVIEW

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Looping Structures

| Loop Type  | Checks Condition                                 | Best For                                                 |
|------------|--------------------------------------------------|----------------------------------------------------------|
| for        | Before each iteration                            | A known, fixed number of repetitions                     |
| while      | Before each iteration                            | Unknown repetitions, condition-driven                    |
| do...while | AFTER each iteration — always runs at least once | When the loop body must run at least one time regardless |

JS

```
// FOR LOOP
for (i = 0; i < 10; i++) {
    document.write("this is line " + i + "<br>");
}

// WHILE LOOP
while (total < 10) {
    ctr++;
    total += values[ctr];
}

// DO-WHILE LOOP
do {
    n++;
```

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
total += values[n];
} while (total < 10);
```

## Functions

A function is a named, reusable block of code. It runs only when CALLED, and can accept input values called **arguments**.

```
function Greet() {
    alert("Hello World!");
}

// with an argument:
function Greet(who) {
    alert("Greetings, " + who);
}

Greet("Kian"); // -> "Greetings, Kian"
```

## Objects — Properties & Methods

An **object** bundles related data (**properties**) and actions (**methods**) under one name, accessed with dot notation.

| Concept  | Example                      | Meaning                         |
|----------|------------------------------|---------------------------------|
| Property | <code>txtGreet.length</code> | A stored value belonging to the |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Concept | Example                         | Meaning                                                                                    |
|---------|---------------------------------|--------------------------------------------------------------------------------------------|
|         |                                 | object —<br>no <code>()</code>                                                             |
| Method  | <code>txtGreet.indexOf()</code> | An action<br>the<br>object<br>can<br>perform<br>— always<br>called<br>with <code>()</code> |

Creating a Custom Object

JS

```
var person = new Object();
person.firstname = "Juan";
person.lastname = "Santos";
person.age = 19;

var x = person.age; // x now holds 19
```

Built-In Objects — String Objects

JS

```
var string1 = new Object();
string1.prop = "I am a property";
alert(string1.prop);
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Forward Link — PF 101

Objects with properties and methods here are your first exposure to **Object-Oriented Programming** — the entire subject of **PF 101** (Year 2). A JS object's "properties" map to a Java class's "fields," and its "methods" map directly to Java class methods.

## CHAPTER 12

# Regular Expressions, Events & Embedding JavaScript

This closing JS chapter covers pattern-matching text with regular expressions, making pages react to user actions with events, and the different ways JavaScript code actually attaches to an HTML page.

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Regular Expressions  
(Regex)

A regular expression is a search PATTERN used to match, find, or replace text.

JS

```
// object form:
var regexPatt = new RegExp(patter
// shorthand/literal form (more
var regexPatt = /pattern/modifie
```

Modifiers

| Modifier | Effect                                                  |
|----------|---------------------------------------------------------|
| i        | Case-insensitive matching                               |
| g        | Global matching — finds ALL matches, not just the first |
| m        | Multiline matching                                      |

Patterns

| Pattern | Type                    | Matches                                           |
|---------|-------------------------|---------------------------------------------------|
| /abc/   | Simple pattern          | The literal characters "abc" anywhere in a string |
| /ab*c/  | Special character ( * = | "ac", "abc", "abbbbc" — any                       |

## 16    Controllers, Blade &amp; Migrations

| Method                  | Belongs To    | Returns                                                       |
|-------------------------|---------------|---------------------------------------------------------------|
| <code>.match()</code>   | String        | Array of matches                                              |
| <code>.search()</code>  | String        | Index position of first match                                 |
| <code>.replace()</code> | String        | New string with matches replaced                              |
| <code>.split()</code>   | String        | Array split at each match                                     |
| <code>.exec()</code>    | RegExp object | Match details, or <code>null</code>                           |
| <code>.test()</code>    | RegExp object | <code>true</code> / <code>false</code> — does it match at all |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
var str = "The rain in Spain sta
var res = str.match(/ain/g);
// res -> ["ain", "ain", "ain"]
```

## RegExp Object Reference

| Category   | Members                                                                                            |
|------------|----------------------------------------------------------------------------------------------------|
| Methods    | <code>exec()</code> , <code>test()</code>                                                          |
| Properties | <code>source</code> , <code>global</code> ,<br><code>ignoreCase</code> ,<br><code>multiline</code> |

## Events & Event Handlers

| Event Category      | Examples                                                                        |
|---------------------|---------------------------------------------------------------------------------|
| Mouse events        | <code>onclick</code> ,<br><code>onmouseover</code> ,<br><code>onmouseout</code> |
| Keyboard events     | <code>onkeydown</code> ,<br><code>onkeyup</code>                                |
| Frame/Object events | <code>onload</code> ,<br><code>onresize</code>                                  |
| Form events         | <code>onsubmit</code> ,<br><code>onchange</code> ,<br><code>onfocus</code>      |

## Live: Event Handler in Action



HTML

Web Systems & Technologies  
1

Complete Study Guide · AY 2026-2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
<button onclick="document.write(Date())">
  Check Time
</button>
```

Gotcha — alert() in  
Sandboxed Previews

Popup dialogs like `alert()`, `confirm()`, and `prompt()` are often blocked inside sandboxed iframes (like this guide's live previews) for security reasons — but work normally in a real browser tab or VS Code Live Server. Also note: calling `document.write()` from an event handler AFTER the page has loaded erases the whole page rather than appending to it — that's why the live demo above updates an element's `innerHTML` instead, which is the safer real-world pattern.

Popup Boxes

| Type      | Function                      | Returns                                  |
|-----------|-------------------------------|------------------------------------------|
| Alert Box | <code>alert("message")</code> | Nothing — just displays and waits for OK |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Type                         | Function                        | Returns                                       |
|------------------------------|---------------------------------|-----------------------------------------------|
| Confirm Box<br>supplementary | confirm("message")              | true<br>(OK) or<br>false<br>(Cancel)          |
| Prompt Box<br>supplementary  | prompt("message",<br>"default") | The text<br>typed, or<br>null if<br>cancelled |

Confirm and Prompt aren't on the lecture slides, but they're the other two standard JS popup types alongside Alert, and follow the exact same calling pattern — worth knowing as a set.

Client-Side JavaScript  
— Core Objects

| Object          | Represents                                                  |
|-----------------|-------------------------------------------------------------|
| window          | The entire browser window/tab                               |
| location        | The current page's URL                                      |
| document        | The current page's content (the DOM)                        |
| Element objects | Individual tags on the page (a specific button, input, div) |



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

# Embedding JavaScript in HTML — 4 Ways

| Method                              | Example                                                                  |
|-------------------------------------|--------------------------------------------------------------------------|
| 1. Inline scripting                 | <code>&lt;script&gt; /* code */ &lt;/script&gt;</code>                   |
| 2. External scripting               | <code>&lt;script src="scriptLocation.js"&gt;script&gt;</code>            |
| 3. Via HTML event handler attribute | <code>&lt;input onchange="order.giftwrap : this.checked;"&gt;</code>     |
| 4. Via javascript: protocol         | <code>&lt;a href="javascript:alert('Hi')"&gt;Click here&lt;/a&gt;</code> |

### Best Practice

Method 2 (external `.js` file) is standard for real projects — same reasoning as external CSS: separation of concerns, reusability across pages, and browser caching. Methods 3 and 4 mix JS directly into HTML attributes and are considered messy by modern standards, even though they still work and appear on your slides.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Where This Guide Goes Next

HTML (Weeks 2–3) + CSS (Week 4) + JavaScript (Weeks 5–6) cover the front-end half of WS 101 and are the midterm material. The course then moves to server-side work for finals: relational database design and SQL (Weeks 10–11, covered in Chapters 13–14) and the Laravel framework (Weeks 12–14, covered in Chapters 15–16). Weeks 7–9 are the midterm exam period and aren't part of either source deck.

## CHAPTER 13

# Relational Model, ER Diagrams & UML

Weeks 10–11 shift WS 101 from front-end code to **database design** — how data gets organized before any PHP or SQL touches it. This chapter covers the theory: hierarchical

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

structures, the relational model, ER diagrams, and UML. Chapter 14 covers writing the actual SQL and connecting PHP to it.

## Hierarchical Data Structures

Before relational databases became the standard, data was modeled as a **hierarchical structure** — an upside-down tree showing parent-child relationships. It's composed of **segments** (the equivalent of a file system's record type), each of which can have a **parent** and **children**. The topmost segment is the **root segment**, with everything below it organized into levels (Level 1, Level 2, Level 3...).

| Term              | Meaning                                                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Hierarchy chart   | A diagram created to document a program or database — conveys the big picture of the modules used, or the information a database contains |
| Hierarchical path | An ordered sequencing of                                                                                                                  |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Term

### Meaning

segments tracing the hierarchical structure

Preorder traversal

Also called the *hierarchical sequence* or *left-list path* — traces every segment starting from the root, beginning at the leftmost segment first

### Why This Still Matters

Even though most modern databases are relational (tables, not trees), hierarchy charts and left-to-right traversal thinking still show up constantly — file system folders, HTML's DOM tree, JSON nesting, and org charts all follow the same parent → children logic.

## The Relational Model

**Dr. E.F. Codd**, a British scientist working for IBM, invented the **relational model** for database management. Instead of trees, it represents a database as a collection of **relations** — a formal name for **tables** made up of rows and columns.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

| Relational Term | Everyday Name | Example |
|-----------------|---------------|---------|
|-----------------|---------------|---------|

|          |       |          |
|----------|-------|----------|
| Relation | Table | Employee |
|----------|-------|----------|

|           |                |                    |
|-----------|----------------|--------------------|
| Attribute | Column / field | Emp_Name , Emp_Age |
|-----------|----------------|--------------------|

|       |              |                          |
|-------|--------------|--------------------------|
| Tuple | Row / record | One employee's full data |
|-------|--------------|--------------------------|

|             |                       |        |
|-------------|-----------------------|--------|
| Primary Key | Unique row identifier | Emp_ID |
|-------------|-----------------------|--------|

| Emp_ID (PK) | Emp_Name | Emp_Address | Emp_Age |
|-------------|----------|-------------|---------|
| 00001       | Melanie  | Novaliches  | 27      |
| 00002       | Michael  | Cainta      | 26      |
| 00003       | Phoebe   | Marikina    | 20      |
| 00004       | Lomel    | Taguig      | 28      |

## Relating Tables — Foreign Keys

A single table can only hold so much before it gets messy.

Relational design splits data across multiple tables, then reconnects them using a **foreign key** — a column that references another table's primary key. A *Sales-summary* table can link an `Employee` table to an `Items`

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

table without duplicating employee or item details inside it.

| Table         | Role                                   | Key Columns                 |
|---------------|----------------------------------------|-----------------------------|
| Employee      | Stores employee records                | Emp_ID (PK)                 |
| Items         | Stores item records                    | Item_ID (PK)                |
| Sales-summary | Links employees to the items they sold | Emp_ID , Item_ID (both FKs) |

The Payoff

If Melanie's address changes, you update *one* row in **Employee** — every sale record referencing her **Emp\_ID** stays accurate automatically. That's the core reason relational design beats one giant flat table.

Entity-Relationship  
(ER) Modeling

Before building tables, designers sketch an **Entity-Relationship (ER) Diagram** — a graphical

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

way to express a database's overall logical structure. The classic teaching example: two **PERSON** entities playing the **roles** of **Husband** and **Wife**, connected by the relationship "**is married to.**" Nouns become entities; verbs become relationships.

ER Diagram Notation

| Symbol           | Component     | Meaning                                                                          |
|------------------|---------------|----------------------------------------------------------------------------------|
| Rectangle        | Entity        | A distinct object or thing (a person, a course, a product)                       |
| Double rectangle | Weak entity   | An entity that can't be uniquely identified without another entity it depends on |
| Underlined oval  | Key attribute | The attribute that uniquely identifies each instance                             |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Symbol

Component

Meaning

(e.g.  
StudentNo )

Double  
oval

Multivalued  
attribute

An attribute  
that can  
hold more  
than one  
value at  
once

Dashed  
oval

Derived  
attribute

A value  
calculated  
from other  
data rather  
than stored  
directly (e.g.  
Age from  
Birthdate)

Diamond

Relationship

How two  
entities  
connect  
(e.g. *studies*,  
*enrolls in*)

Cardinality — Crow's  
Foot Notation

**Cardinality** describes how many instances of one entity can relate to instances of another. The most common notation style is **crow's foot**, drawn at the end of the connecting line closest to the entity it describes.



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Notation

### Meaning

One to one

Exactly one record relates to exactly one record on the other side

One to many (mandatory)

One record relates to at least one, possibly many, records on the other side

Many

Multiple records on this side can relate to the other entity

One or more (mandatory)

At least one relationship must exist

One and only one (mandatory)

Exactly one relationship is required — no more, no fewer

Zero or one (optional)

The relationship may or may not exist, but never more than one

Zero or many (optional)

The relationship is fully optional, and can repeat

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

## Reading a Diagram — Worked Examples

### 1 Student — Class

A many-to-many relationship: a Student can take many Classes, and a Class can hold many Students.

### 2 Student — Class — Instructor

Adds a third entity: each Class also connects to one Instructor, forming a chain rather than a single pair.

### 3 Student — Subject

Many-to-many, crow's feet on both ends: Students study many Subjects, and each Subject is studied by many Students.

### 4 Student entity with attributes

StudentNo (underlined = key attribute), StudName, Address, ContactNo attach directly to the Student rectangle.

### 5 Student *studies* Course — the full pattern

Student ( StudentNo, StudName, Address, ContactNo ) connects through the *studies* relationship diamond to Course ( CourseCode, CourseTitle ) — two entities, their attributes, and the relationship joining them.

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Unified Modeling  
Language (UML)

**UML** is a broader modeling approach than ER diagrams — it models many aspects of software development for object-oriented systems, not just the database layer.

| Diagram Type | What It Shows                                                                                                     |
|--------------|-------------------------------------------------------------------------------------------------------------------|
| Class        | A static structure diagram — a system's classes, their attributes, methods, and the relationships between classes |
| Use Case     | How the system behaves from the end user's point of view                                                          |
| State        | The possible states an object can be in                                                                           |
| Sequence     | The sequence of interactions between objects over time                                                            |
| Activity     | The business and operational step-by-step workflow                                                                |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Diagram  
Type

What It Shows

of a system's  
components

Component

How a system's  
physical  
components are  
organized and  
connected

ER Diagram vs. Class  
Diagram — Don't Mix  
Them Up

Both use rectangles and lines, which makes them easy to confuse. An **ER diagram** models *data* — what tables and columns exist and how they relate. A UML **Class diagram** models *code* — what classes, methods, and objects exist and how they relate. ER comes first, when you're designing the database; Class diagrams come later, when you're designing the application logic that will use it.

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

# SQL & Connecting PHP to MySQL

## Structured Query

## Language — Three Components

SQL (Structured Query Language) is how you talk to a relational database. It splits into three sub-languages, each with a distinct job:

| Component                   | Full Name                  | Job                                                                                                                                 |
|-----------------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| DML<br><div>most used</div> | Data Manipulation Language | Retrieve, update, add, or delete data <i>inside</i> a database<br>( <b>SELECT</b> , <b>INSERT</b> , <b>UPDATE</b> , <b>DELETE</b> ) |
| DDL                         | Data Definition Language   | Create and modify the database structure itself<br>( <b>CREATE</b> , <b>ALTER</b> , <b>DROP</b> )                                   |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026-2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Component

Full Name

Job

DCL

Data Control  
Language

Maintain  
proper  
security  
and  
permissions  
for the  
database  
( GRANT ,  
REVOKE )

Data Definition —  
Creating & Modifying  
Structure

```
SQL

-- create and delete a database
CREATE DATABASE DBEnrollment;
DROP DATABASE DBEnrollment;

-- create a table
CREATE TABLE tblStudent (
    studentNumber INT PRIMARY KEY,
    studentName VARCHAR(50),
    Course VARCHAR(20)
);

-- add a column to an existing table
ALTER TABLE tblStudent ADD sectionNumber INT;

-- remove a column from an existing table
ALTER TABLE tblStudent DROP COLUMN sectionNumber;

-- delete an entire table
DROP TABLE tblStudent;
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

## Basic Data Retrieval — SELECT

Three clauses do almost all the work of reading data back out of a table:

| Clause        | Job                                                                   |
|---------------|-----------------------------------------------------------------------|
| <b>SELECT</b> | Lists which attributes (columns) you want in the result               |
| <b>FROM</b>   | Lists which table(s) to scan                                          |
| <b>WHERE</b>  | Filters rows using a condition on the attributes named in <b>FROM</b> |

```
SQL

SELECT column_name FROM table_name

-- or select every column:
SELECT * FROM table_name;
```

## Modifying Table Data — INSERT, UPDATE, DELETE

| Statement     | Job                        |
|---------------|----------------------------|
| <b>INSERT</b> | Adds new rows into a table |

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

Statement

Job

UPDATE

Edits existing data in specific rows and columns

DELETE

Removes entire rows



SQL

```
INSERT INTO tblStudent (LastName, FirstName, CourseCode)
VALUES ('Reyes', 'Melanie', 'BSIT')
```

```
UPDATE tblStudent SET CourseCode = 'BSED' WHERE StudentID = 1
```

```
DELETE FROM tblStudent WHERE StudentID = 1
```

Fixing a Slide Typo — Multi-Row INSERT

The lecture slide's INSERT example crams two student records into one SQL statement by using the `VALUES()` clause with 3 columns but 6 values — too many for the columns. To insert Reyes/Melanie *and* Smith/John in one statement, you must use two separate parenthesized groups instead:

```
INSERT INTO tblStudent (LastName, FirstName, CourseCode)
VALUES ('Reyes', 'Melanie', 'BSIT'),
      ('Smith', 'John', 'BSED');
```

MySQL Data Types

Type

Description

INT

Whole numbers, roughly -2,147,483,648 to 2,147,483,647

DECIMAL

Floating-point / decimal numbers



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Type

### Description

CHAR

Fixed-length string, max 255 characters

VARCHAR

Variable-length string, max 255 characters

TEXT

Long string, max 65,535 characters

DATE

YYYY-MM-DD

TIME

HH:MM:SS

DATETIME

YYYY-MM-DD

HH:MM:SS

combined

## Connecting PHP to MySQL

PHP talks to MySQL through the `mysqli_connect()` function, which needs four pieces of information:

### Parameter

### Description

Host

A hostname or IP address — usually `"localhost"` in development

Username

The MySQL account username — often `"root"` on a local Xampp install

Web Systems & Technologies  
1

Complete Study Guide · AY 2026–2027

INTRODUCTION

00 Overview & How the Web Works

HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

ParameterDescription

PasswordThe MySQL account password

DatabaseThe name of the specific database to use

```
<?php
$dbServername = "localhost";
$dbUsername   = "root";
$dbPassword   = "";
$dbName       = "qcu";

$conn = mysqli_connect($dbServer
?>
```

Centralizing the Connection

Rather than repeating the connection code on every page, save it once (commonly as `includes/dbh.php`, short for "database handler") and pull it into any page that needs it:

```
<?php
include_once 'includes/dbh.php';
?>
<!DOCTYPE html>
<html>
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

```
<head><title></title></head>
...
```

### Why include\_once, Not include

`include_once` guarantees the file is only pulled in once, even if several other included files also try to include it — this prevents "function already defined" fatal errors from a connection script loading twice.

## Reading Data — SELECT via PHP



PHP

```
<?php
include_once 'includes/db.php';
?>
<!DOCTYPE html>
<html>
<head><title></title></head>
<body>

<?php
    $sql = "SELECT * FROM stud;";
    $result = mysqli_query($conn,
    $resultCheck = mysqli_num_rows

    if ($resultCheck > 0) {
        while ($row = mysqli_fetch_a
            echo $row['stud_uid'] . "<
        }
    }
?>
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

&lt;/body&gt;

&lt;/html&gt;

### 1 **mysqli\_query(\$conn, \$sql)**

Sends the SQL string to the database over the connection and returns a result set.

### 2 **mysqli\_num\_rows(\$result)**

Counts how many rows came back — used here to check "did we actually get anything?" before looping.

### 3 **mysqli\_fetch\_assoc(\$result)**

Pulls one row at a time as an associative array (`$row['column_name']`). The `while` loop keeps calling it until no rows are left, at which point it returns `false` and the loop ends.

## Writing Data — INSERT via PHP



PHP

```
<body>
<?php
    $sql = "INSERT INTO stud (stud
        VALUES ('Paolo', 'Lope
    $result = mysqli_query($conn,
?>
</body>
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

### Beyond the Slides — SQL Injection

Building queries by concatenating raw variables straight into the SQL string, as shown above, is exactly how SQL injection attacks work — if `$sql` ever includes unfiltered user input (like a login form field) instead of hardcoded values, an attacker can inject their own SQL. Production code uses **prepared statements** (`mysqli_prepare()` with `?` placeholders, or PDO) instead. It's not on this deck, but it's the single most important upgrade to make once you're past the fundamentals.

## CHAPTER 15

# Laravel Fundamentals — Installation, Structure & Routing

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

Weeks 12–14 introduce **Laravel**, a PHP framework that automates much of the plumbing raw PHP+MySQL requires — routing, templating, and database schema management. This chapter covers install and routing (Week 12); Chapter 16 covers controllers, Blade views, and database migrations (Weeks 13–14).

## What Is Laravel?

Laravel is a PHP framework — a large, pre-built collection of tools and conventions that handles the repetitive plumbing of a web application so you write far less code than raw PHP requires.

| Benefit          | Why It Matters                                                                    |
|------------------|-----------------------------------------------------------------------------------|
| Quick and simple | Built-in tools (routing, ORM, templating) replace hundreds of lines of manual PHP |
| Security         | Built-in protection against common attacks, including SQL injection and CSRF      |

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### Benefit

### Why It Matters

Better  
performance

Caching and  
optimization  
tools ship with  
the framework

Traffic  
handling

Designed to  
scale for high-  
traffic  
applications

Flexible

Not opinionated  
about how you  
structure  
business logic

Third-party  
integrations

Straightforward  
to plug in  
payment  
gateways, mail  
services, etc.

Simple  
maintenance

Consistent  
structure makes  
it easier for other  
developers to  
pick up your  
code

Cost

Free and open-  
source — no  
licensing fees

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

# Installing Laravel — Prerequisites

## 1 Xampp Server (5.0+)

Provides Apache (web server), MySQL (database), and PHP in one local package.

## 2 PHP (5.5+)

Bundled with Xampp — Laravel needs a minimum PHP version to run.

## 3 Composer

PHP's package manager, downloaded from [getcomposer.org](https://getcomposer.org). During setup, point it to your Xampp PHP install (e.g. `xampp/php/php.exe`).

# Creating a Laravel Project

## 1 Open a terminal and verify Composer

Type `composer` and press Enter — a working install prints its version and a list of commands.

## 2 Navigate to htdocs

Xampp serves web files from its `htdocs` folder — change into it, e.g. `cd xampp/htdocs` (adjust the drive/path to match your install).

## 3 Create the project

Composer downloads Laravel and scaffolds a new project folder here.



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &  
Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &  
UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

4

## Open it in the browser

Visit `https://localhost/`

- ▲ `project_name/public` — Laravel's default welcome page confirms a working install.

### Legacy Version Note

This deck's screenshots show **Laravel 5** (circa 2016). Current Laravel installs differently ( `composer create-project laravel/laravel project-name` , or the `laravel new` CLI installer) and typically serves via `php artisan serve` rather than a manually configured Xampp `htdocs` folder. The *concepts* in this and the next chapter — routing, controllers, Blade, migrations — are still accurate; only a handful of install commands and filenames have moved since Laravel 5.

## Routing — The Static get() Method

**Routing** is the concept of mapping a URI (like `/hello` ) to a destination — a page, a piece of logic, or a controller method. In Laravel 5, routes were defined in `app/Http/routes.php` :



PHP

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

```
Route::get('/', function () {
    return view('welcome');
});
```

`Route` is a class with a static method `get()` that takes a URI and a closure (an anonymous function). The closure runs `view('welcome')`, which returns the contents of `welcome.blade.php` — what a visitor sees when they load the site's home page.

## Adding a New Route

Any new page starts the same way — call `Route::get()` with a new URI and a closure that returns something to display:

```
Route::get('/hello', function () {
    echo "<html><head><title>La
    <style>
        body { font-family: 'Lato'
        .title { font-size: 110px;
    </style></head>
    <body>
        <div class='title'>Hello,
        <p>How are you? I hope you
    </body></html>";
});
```

Visiting `http://localhost:8000/hello` now renders the echoed HTML directly. This works for a

## Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

### INTRODUCTION

- 00 Overview & How the Web Works

### HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

### CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

### JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

### DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

### LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

quick demo, but hand-writing HTML inside a PHP closure doesn't scale — that's exactly the problem Blade views (Chapter 16) solve.

### CHAPTER 16

# Laravel MVC — Controllers, Views/Blade & Database Migration

## The Controller's Role in MVC

Laravel follows the **MVC (Model-View-Controller)** pattern. The **controller** sits in the middle — it controls the flow between the Model (data) and the View (presentation). A controller is best thought of as a **transporter**: it executes logic and returns a view, but never needs to know what that view contains or how it's displayed.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

## Creating a Controller

Controllers live in `app/Http/Controllers`. Here's a minimal one that just returns a view:

```
<?php
// app/Http/Controllers/MyController.php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Http\Requests;

class MyController extends Controller
{
    // returning a simple page view
    public function returningASimplePage()
    {
        return view('newpage');
    }
}
```

## Routing to a Controller Method

Once a controller exists, the route no longer holds a closure — it points to the controller and the method:

```
// routes.php
Route::get('newpage', 'MyController@returningASimplePage');
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

Visiting `http://localhost:8000/newpage` now runs `MyController`'s `returningASimplePage()` method, which returns the `newpage` view.

### Legacy Syntax — 'Controller@method'

The

`'MyController@returningASimplePage'` string syntax was removed in Laravel 8. Current Laravel uses an array-callable form instead: `Route::get('newpage', [MyController::class, 'returningASimplePage'])`. Use the array form on a modern Laravel version even though these slides show the older string form.

## Controller Rules to Remember

### 1 Always use public functions

Laravel's router can only call methods it can access from outside the class — `public function index() { return view('home'); }`

### 2 Route the controller, not a closure

Once a controller exists for a route, stop using inline anonymous functions — route to the controller method instead:

```
Route::get('home',
'MyController@index');
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

## Views & Blade — The Master Layout

Laravel's templating engine is called **Blade**. Rather than repeating the same HTML skeleton on every page, you build one **master layout** with `@yield()` placeholders that child views fill in:

```
BLADE

{{-- resources/views/pages/master.blade.php --}}
<!DOCTYPE html>
<html>
<head>
    @yield('head')
    <link rel='stylesheet' href='...'>
    <title>@yield('title')</title>
</head>
<body>
    <div class="container">
        <div class="heading">@yield('heading')</div>
        <div class="content">@yield('content')</div>
        <div class="footer">@yield('footer')</div>
    </div>
</body>
</html>
```

## Child Views — @extends & @section

A child view declares which master it extends, then fills each named section between `@section('name')` and `@stop`:

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations



BLADE

```
{{-- resources/views/pages/about
@extends('pages.master')

@section('title')
    It's a test page
@stop

@section('heading')
    {{ $name }} {{ $profession }}
@stop

@section('content')
    <div class='h1'>Every Writing
    <p>There was a problem few mir
@stop

@section('footer')
    Home of Sanjib Sinha
@stop
```

The master page plays the role of a **parent**; the child view ( `about.blade.php` ) fills in its blanks. Logic flows down from parent to child.

## Blade Echo Syntax — Double Curly Braces

The `{{ $variable }}` syntax is Blade's shorthand for `echo`. It's not just cosmetic — Blade automatically runs the output through `htmlspecialchars()`, escaping any HTML/script characters before printing them. That's a built-in defense against

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

cross-site scripting (XSS) that raw `echo $variable;` doesn't give you for free.

## Passing Data — Controller to View

Three equivalent ways to hand variables from a controller method to its view:

```
PHP

// app/Http/Controllers/MyController.php

// Method 1 -- compact()
public function about() {
    $name = 'John Doe';
    $profession = 'A Writer';
    return view('pages.about', compact('name', 'profession'));
}

// Method 2 -- chained with()
public function about() {
    $name = 'John Doe';
    return view('pages.about')->with('name', $name);
}

// Method 3 -- array with([...])
public function about() {
    return view('pages.about')->with([
        'name' => 'John Doe',
        'profession' => 'A Writer'
    ]);
}
```



# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview & How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links & Images

03 Tables & Forms

04 Multimedia & Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display & Position

07 Typography, Color & BG

08 Flexbox

09 Grid, Units & Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions & Objects

12 Regex, Events & Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams & UML

14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure & Routing

16 Controllers, Blade & Migrations

### compact() Explained

```
compact('name',  
'profession')
```

automatically builds  

```
['name' => $name,  
'profession' =>  
$profession]
```

from variables already in scope — a shortcut for Method 3 when your array keys match your variable names exactly.

## The .env File & Environment Config

Database credentials live in a project-root `.env` file, referenced by `config/database.php`:

```
# .env  
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=testdb  
DB_USERNAME=root  
DB_PASSWORD=pass
```

## Keeping Secrets Safe — .gitignore

If this project is ever pushed to GitHub or another cloud repo, the `.env` file (and other sensitive files) should never go with it.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

Laravel ships with them pre-listed in `.gitignore`:



GITIGNORE

```
# .gitignore
/vendor
/node_modules
/public/storage
Homestead.yaml
Homestead.json
.env
```

### Double-Check Before Every Push

A committed `.env` file exposes real database passwords in your repo's history — deleting it in a later commit doesn't remove it from history. Confirm `.env` is listed in `.gitignore` before your first commit, not after.

## Database Migration — What & Why

Before migrations, creating a database, table, and columns meant hand-writing SQL or clicking through phpMyAdmin — repetitive and hard to keep in sync across a team. **Migrations** let you define a schema as PHP code, version it alongside the rest of your project, and run it with one command.

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

# Anatomy of a Migration File

Laravel ships with a migration that creates the default `users` table — a good reference for the pattern:

```
<?php
// database/migrations/..._create_users_table.php
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Database\Migrations\Migration;

class CreateUsersTable extends Migration
{
    public function up()
    {
        Schema::create('users', function (Blueprint $table) {
            $table->increments('id');
            $table->string('name', 255);
            $table->string('email', 255);
            $table->string('password', 60);
            $table->rememberToken();
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::drop('users');
    }
}
```

### Piece

`up()`

### Job

Runs when  
the migration

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

### Piece

### Job

is applied —  
defines what  
to build

`down()`

Runs when  
the migration  
is rolled back  
— undoes  
what `up()`  
did

`Schema::create()`

Creates a  
new table  
using a  
`Blueprint`

`$table->increments('id')`

Auto-  
incrementing  
primary key  
column

`$table->timestamps()`

Adds  
`created_at`  
and  
`updated_at`  
columns  
automatically

## Migration Workflow — Create, Run, Update

1

### Generate a migration file

```
php artisan make:migration  
create_tasks_table --  
create="tasks" — scaffolds a  
new timestamped file in  
database/migrations.
```

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

00 Overview &amp; How the Web Works

## HTML

01 Document Structure

02 Text, Lists, Links &amp; Images

03 Tables &amp; Forms

04 Multimedia &amp; Attributes

## CSS

05 CSS Fundamentals

06 Box Model, Display &amp; Position

07 Typography, Color &amp; BG

08 Flexbox

09 Grid, Units &amp; Responsive

## JAVASCRIPT

10 JS Fundamentals

11 Control Structures, Functions &amp; Objects

12 Regex, Events &amp; Embedding JS

## DATABASES & BACKEND

13 Relational Model, ER Diagrams &amp; UML

14 SQL &amp; Connecting PHP to MySQL

## LARAVEL FRAMEWORK

15 Installation, Structure &amp; Routing

16 Controllers, Blade &amp; Migrations

### 2 Define the columns inside up()

Edit the generated file — e.g. add `$table->string('title');` `$table->text('body');` inside `Schema::create()`.

### 3 Run the migration

`php artisan migrate` — applies every pending migration and creates the actual MySQL tables.

### 4 Verify in phpMyAdmin

The new table (and its columns) now appears in the actual database — no manual SQL was typed.

## Adding a Column After the Fact

Forgot a column after the table's already built? Don't edit the old migration — generate a new one that targets the existing table with `--table` instead of

`--create :`

BASH

```
php artisan make:migration add_
```

Then add `$table->text('reminder')->nullable();` inside its `up()`, and run `php artisan migrate` again — Laravel only applies migrations it hasn't run yet, so this adds the new

# Web Systems & Technologies 1

Complete Study Guide · AY 2026–2027

## INTRODUCTION

- 00 Overview & How the Web Works

## HTML

- 01 Document Structure
- 02 Text, Lists, Links & Images
- 03 Tables & Forms
- 04 Multimedia & Attributes

## CSS

- 05 CSS Fundamentals
- 06 Box Model, Display & Position
- 07 Typography, Color & BG
- 08 Flexbox
- 09 Grid, Units & Responsive

## JAVASCRIPT

- 10 JS Fundamentals
- 11 Control Structures, Functions & Objects
- 12 Regex, Events & Embedding JS

## DATABASES & BACKEND

- 13 Relational Model, ER Diagrams & UML
- 14 SQL & Connecting PHP to MySQL

## LARAVEL FRAMEWORK

- 15 Installation, Structure & Routing
- 16 Controllers, Blade & Migrations

column without touching or re-creating the rest of the table.

### In Summary

Database migration is a technique to create, update, or delete a table and its columns using version-controlled PHP files instead of raw SQL or a GUI tool. On a team working from different machines, everyone runs the same migration files and ends up with an identical database schema — no more "it works on my database" bugs.

**WS 101 — Web Systems and Technologies 1** · Complete Study Guide  
QCU Academic OS · Compiled by  
KianoRoku18Ino · [github.com/](https://github.com/KianoRoku18Ino)  
KianoRoku18Ino · AY 2026–2027  
QCU Academic OS v4.1 · Semester Zero  
Advance Prep Material · Live Demos  
Render Locally, No Internet Required